INSTITUTO SUPERIOR TECNOLÓGICO QUITO
Formamos tu PROPÓSITO DE VIDA

SISTEMA MULTIEMPRESA DE GESTIÓN VETERINARIA

ARQUITECTURA Y API REST

itq

LUIS ALMACHI · EMILIO LÓPEZ · CRISTOPHER SANTANDER
SEBASTIÁN GONZAGA · ÁNGEL GALARZA

TUTOR: M.Sc. ELVIS PACHACAMA

INSTITUTO SUPERIOR TECNOLÓGICO QUITO

CONTROL DE VERSIÓN: 05 NOVIEMBRE 2026

MANUAL TÉCNICO DE ARQUITECTURA Y API

Diseño, datos, seguridad e integración · versión 2.0

Proyecto: ControlPets / Sistema multiempresa de gestión veterinaria | Revisión documental: 05 de noviembre de 2026

Institución: Instituto Superior Tecnológico Quito (ITQ). Documento académico de referencia para revisión y mantenimiento del proyecto.

Guía de contenidos

01 Visión y arquitectura

02 Estructura de proyecto

03 Entidades y persistencia

04 Autenticación y multiempresa

05 API REST

06 Controles de calidad

07 Evolución técnica

Los números de sección facilitan la consulta. La paginación puede variar al editar esta versión Word.

1. Visión y arquitectura

La solución separa una SPA React y una API Java Jakarta Servlet expuesta a través de HTTPS. La lógica de negocio no se ejecuta en Vercel: reside en el servidor Tomcat sobre Render; los datos se almacenan en PostgreSQL de Render.

```
USUARIO → VERCEL [React/TypeScript]
        ↓ HTTPS + JWT
      RENDER [Tomcat 10.1 / API Java]
        ↓ filtro → servlet → service → DAO (JDBC/HikariCP)
      RENDER [PostgreSQL 18 / controlpets]
```

Componente	Responsabilidad	Ubicación en repo
SPA	Navegación, formularios, vistas y sesión	Frontend/src/app y src/features
API	Controladores de recursos HTTP	Backend/src/main/java/com/itq/servlet
Servicios	Reglas, validaciones y transacciones	.../service
Persistencia	SQL y mapeo de entidades	.../dao y .../model
Seguridad	JWT, filtros de autenticación, permisos	.../filter, .../security, .../util
Migraciones	Evolución del esquema	Backend/src/main/resources/sql

2. Directorios y dependencias principales

El empaquetado Maven genera VETERINARIAITQ.war. Sus dependencias incluyen controlador PostgreSQL JDBC, HikariCP, Gson, java-jwt y jBCrypt. La SPA utiliza Vite y React Router; el contrato compartido con la API sigue el formato {exito, mensaje, datos}.

```
cd Backend && mvn clean package
cd ../Frontend && npm install && npm run build
```

3. Entidades y persistencia relacional

El respaldo efectivamente restaurado en Render contiene 24 tablas. Algunas entidades relevantes son empresa, usuario, rol, permiso, rol_permiso, cliente, mascota, cita, veterinario, historial_clinico, receta, vacuna, producto, movimiento_inventario, factura y factura_detalle. Verificar el esquema real antes de aplicar nuevas migraciones.

Entidad	Propósito	Relaciones críticas
empresa	Tenant / clínica	Usuarios y recursos asignados.
usuario y rol	Acceso a la plataforma	Rol/permisos; empresa salvo globales.
cliente	Propietario de paciente	Referenciado por mascota.
mascota	Paciente	Cliente, raza, empresa.
cita	Agenda	Mascota, veterinario, fecha/hora.
historial_clinico	Atención clínica	Mascota/profesional.
factura y detalle	Ventas y líneas de cobro	Cliente, productos e importes.
mensualidad_empresa	Cobros SaaS por clínica	Requiere confirmar versión y permisos antes de producción.

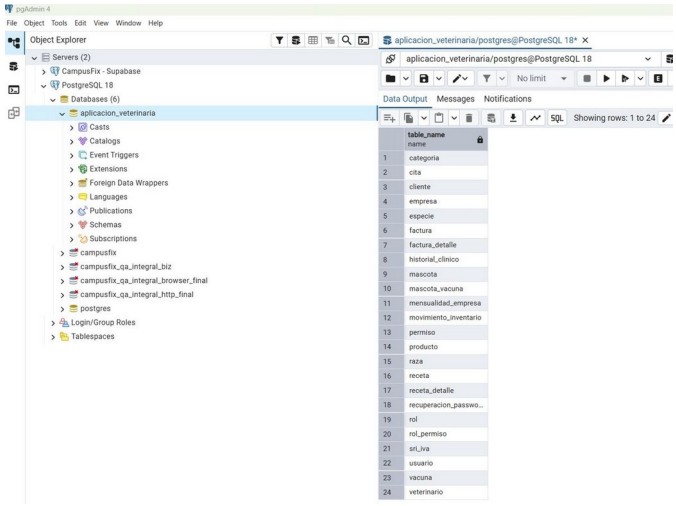


Figura 1. Estructura PostgreSQL verificada localmente.

4. Autenticación, permisos y multiempresa

Las contraseñas de cuentas se comparan con BCrypt y no se guardan en texto plano. El login público es POST /api/auth/login, recibe usuario/clave y entrega datos de sesión/token. Las llamadas protegidas añaden Authorization: Bearer <token>. La política de autorización se aplica en Java, no únicamente ocultando botones en React.

```
POST /VETERINARIAITQ/api/auth/login
{ "usuario": "<USUARIO>", "clave": "<CLAVE>" }

Authorization: Bearer <TOKEN>
```

El identificador de empresa evita que una cuenta operativa consulte registros de otra clínica. Los perfiles SuperUsuario pueden acceder a funciones globales con restricciones distintas. Al crear funcionalidades nuevas, auditar filtros, servicios, DAO y controles del frontend.

5. API REST y manejo de respuestas

Recurso	Operaciones esperables	Observación
/api/auth/login	POST	Público; validación JWT posterior.
/api/empresas	GET, POST, otras	Restricción de SuperUsuario.
/api/usuarios	CRUD según rol	Permisos y estado de cuenta.
/api/clientes y /api/mascotas	CRUD	Filtrado por empresa y validador de datos.
/api/citas	CRUD	Validar disponibilidad profesional/horaria.
/api/productos y /api/movimientos-inventario	CRUD	Actualizar stock coherentemente.
/api/facturas	Consulta y emisión	Integridad de detalle e importes.
/api/mascotas-vacunas	Registro/consulta	No equivale a carné imprimible.

Tabla referencial basada en módulos y servlets; método y ruta exactos por recurso deben confirmarse con @WebServlet/Router del commit desplegado al realizar QA de integración.

6. Pruebas técnicas y cambios verificados

Comprobación	Evidencia disponible
Build local	Maven BUILD SUCCESS, WAR generado.
Local	Tomcat + React + PostgreSQL; login y panel.
PostgreSQL cloud	Restauración 24 tablas desde backup actualizado.
Backend cloud	Render Live, WAR desplegado, servicio Tomcat.
Frontend cloud	Vercel Ready; acceso móvil con SuperUsuario.
Regresión de negocios	Requiere casos independientes por rol/operación.

7. Evolución, control de versiones y deuda técnica

1. Desarrollar y revisar cambios en rama Desarrollo; no publicar secretos ni backups en Git.
2. Generar migración SQL idempotente para cambios de esquema y probarla sobre una copia.
3. Respaldar PostgreSQL Render antes de cada cambio.
4. Actualizar API Render con Manual Deploy cuando se integre código Java.
5. Actualizar SPA Vercel mediante vercel --prod (vinculación manual actual).
6. Ejecutar regresión y actualizar documentación/actas.

Pendientes: ficha/carné imprimible de vacunas, consolidación del módulo de mensualidades y validación ampliada de roles/permisos. Considerar persistencia del hosting de mayor duración tras la evaluación académica.

8. Diccionario funcional del esquema

Tabla / grupo	Campos o datos relevantes	Observación técnica
empresa	Identificación/RUC, razón social, estado	Tenant principal, validador tributario.
usuario	Usuario, clave_hash, rol, activo, estado_cuenta	Seguridad con BCrypt; token JWT en autenticación.
cliente	Identificación, nombres, contactos	Propietario de una o más mascotas.
mascota	Cliente, raza, nombre, fecha_nacimiento	Validación de nombre y fechas.
cita	Mascota, veterinario, fecha_hora, estado	Controlar colisión de turnos.
movimiento_inventario	Producto, cantidad, tipo, fecha	Trazabilidad de entradas y salidas.
factura/factura_detalle	Cabecera, líneas, valor, IVA	Transacción de cobro.
permiso/rol_permiso	Módulo, acción y asignación	RBAC granular.
mensualidad_empresa	Empresa, periodo, valor, vencimiento, estado	Módulo de cobros SaaS con migración; revisar estado real.

9. Reglas de evolución y compatibilidad

Cada cambio de persistencia debe tener una migración repetible o con precondiciones controladas. Se desaconseja modificar directamente el esquema en la instancia de Render sin conservar el SQL en GitHub. El mantenimiento exige comprobar que los nombres de columna y tabla coinciden con los DAOs publicados.

```
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public'
  AND table_name = 'usuario'
ORDER BY ordinal_position;
```

10. Respuestas HTTP y control de excepciones

HTTP	Interpretación	Acción técnica
200/201	Lectura/creación exitosa	Verificar persistencia y respuesta JSON.
204	Eliminación sin contenido	Validar ausencia del registro.
400	Error de entrada	Revisar DTO/validaciones.
401	Ausencia de token o login inválido	Inspeccionar request y JWT/BCrypt.
403	Permisos o estado de empresa	Revisar claims y filtros.
404	Endpoint / recurso no encontrado	Revisar contexto /VETERINARIAITQ y router.
409	Restricción UNIQUE/FK	Registrar conflicto sin exponer SQL sensible.
500	Excepción inesperada	Revisar Logs y traza en backend.

11. Casos mínimos de integración por recurso

El backend y el frontend requieren pruebas de contrato: (a) autenticación, (b) listado autorizado, (c) creación con validación de datos, (d) actualización, (e) borrado o desactivación, (f) aislamiento entre empresas. Las capturas deben incluir URL, respuesta, hora y commit.

12. Análisis de riesgos técnicos

Riesgo	Mitigación recomendada
Credenciales en código/commits	Guardar únicamente placeholders; revocar claves expuestas.
Datos cruzados entre clínicas	Validar id_empresa en filtros, servicios y DAO.
Citas duplicadas	Reforzar reglas concurrentes y prueba de regresión.
Pérdida por plan gratuito	Backups externos antes de vencimiento.
Base sin migraciones completas	Inventario de esquema y scripts versionados.
SMTP no disponible en Free	Desacoplar función o documentar limitación.

13. Anexos institucionales de arquitectura y QA

Fascímiles representativos del manual técnico de arquitectura anterior. Se agregan para conservar evidencia visual de diseño y validación, sin sustituir las especificaciones actualizadas de los apartados precedentes.

Anexo A-06. Registro histórico de arquitectura

INSTITUTO SUPERIOR TECNOLÓGICO QUITO
Formamos tu PROPÓSITO DE VIDA

3. Seguridad y Aislamiento Multiempresa

Seguridad y Aislamiento Multiempresa (Tenants)

Esta sección documenta la implementación de la capa de seguridad, autenticación *Stateless* y las reglas de segregación de datos entre clínicas veterinarias en el backend VETERINARIAITQ.

3.1. Mecanismo de Autenticación (JWT + BCrypt)

La aplicación elimina el manejo de sesiones en memoria en el servidor, adoptando un esquema de tokens **JSON Web Token (JWT)**.

- **Gestión de Contraseñas:** El sistema almacena las claves mediante algoritmos de hashing seguro con **BCrypt**. En la base de datos se registra únicamente el hash correspondiente.
- **Flujo de Autenticación:**
  - El usuario envía sus credenciales al endpoint de login.
  - La capa service consulta al dao correspondiente, valida la existencia del usuario y compara el hash BCrypt.
  - Si la verificación es exitosa, el backend genera un token JWT firmado que empaqueta las propiedades del usuario: id_usuario, id_empresa, rol y lista de permisos.
  - El frontend almacena dicho token y lo adjunta en cada petición HTTP mediante el encabezado:

3.2. Aislamiento de Datos por Empresa (Multitenancy)

El aislamiento de la información entre clínicas veterinarias se ejecuta de forma transparente en la capa de negocio y en la persistencia:

```
[ Cliente / Frontend ]
        | (HTTP Request + Bearer Token)
        ▼
┌──────────────────────────────────────┐
│ Filter (SecurityFilter)              │
│  └─ Valida JWT y extrae: id_empresa, rol, permisos │
└──────────────────────────────────────┘
                  |
                  ▼
```


Extracto: Manual Tecnico Vet.pdf original, página 6.

Anexo A-08. Registro histórico de arquitectura

INSTITUTO SUPERIOR TECNOLÓGICO QUITO
Formamos tu PROPÓSITO DE VIDA

- 403 Forbidden: Devuelto cuando el usuario autenticado intenta acceder a un recurso sin los permisos requeridos para su rol.

4. Diseño de Base de Datos y Persistencia

Diseño de Base de Datos y Diccionario de Datos

Esta sección documenta la persistencia de datos implementada en **PostgreSQL 18** para la base de datos `aplicacion_veterinaria`, la cual consta de 22 tablas relacionales. El diseño garantizó la integridad referencial y el aislamiento multiempresa mediante claves foráneas y tipos de datos normalizados.

4.1. Estrategia de Persistencia y Aislamiento

- **Motor de BD:** PostgreSQL 18.
- **Identificadores Únicos:** Uso de `UUID` (`gen_random_uuid()`) como clave primaria (`PK`) en las tablas transaccionales principales para prevenir la predicción de IDs y facilitar la migración entre entornos.
- **Discriminador Multitenant:** Propagación de la clave foránea `id_empresa` (UUID) en todas las entidades transaccionales para habilitar el filtrado directo a nivel de consultas SQL.

4.2. Diccionario de Datos (Entidades Clave)

Tabla: `empresa`
Almacena la información de cada clínica veterinaria registrada en la plataforma.

Campo	Tipo de Dato	Restricciones	Descripción
`id_empresa`	`UUID`	`PRIMARY KEY`	Identificador único del tenant.
`ruc`	`VARCHAR(13)`	`NOT NULL, UNIQUE`	Registro Único de Contribuyentes.

(+593) 96 356 1961 admisiones@itq.edu.ec Antonio de Ulloa N28-30 y Diego de Atienza (Esq). WWW.ITQ.EDU.EC

Extracto: Manual Tecnico Vet.pdf original, página 8.

Anexo A-11. Registro histórico de arquitectura

INSTITUTO SUPERIOR TECNOLÓGICO QUITO
Formamos tu PROPÓSITO DE VIDA

id_categoria	INTEGER	FK -> categoria(id_categoria)	Categoría del producto.
id_iva	INTEGER	FK -> sri_iva(id_iva)	Porcentaje de IVA aplicable.
nombre	VARCHAR(150)	NOT NULL	Nombre comercial del ítem.
precio_unitario	DECIMAL(10,2)	NOT NULL	Precio de venta al público.
stock_actual	INTEGER	NOT NULL	Inventario físico existente.
stock_minimo	INTEGER	NOT NULL	Límite para disparo de alertas
fecha_caducidad	DATE	NULLABLE	Vencimiento del medicamento

5. Especificación de Endpoints y Contrato de API

Control de Calidad y Reporte de Testing (QA)

En esta sección se detallan las pruebas de verificación funcionales ejecutadas sobre el entorno de despliegue para validar la conectividad, la persistencia en la base de datos PostgreSQL, el empaquetamiento en Tomcat y las respuestas de la API REST.

5.1. Entorno de Ejecución y Herramientas

- **Servidor de Aplicaciones:** Apache Tomcat 10.1.55.
- **Base de Datos:** PostgreSQL 18 (Base: `aplicacion_veterinaria`, 22 tablas mapeadas).

(+593) 96 356 1961 admisiones@itq.edu.ec Antonio de Ulloa N28-30 y Diego de Atienza (Esq). WWW.ITQ.EDU.EC

Extracto: Manual Tecnico Vet.pdf original, página 11.

Anexo A-13. Registro histórico de arquitectura

INSTITUTO SUPERIOR TECNOLÓGICO QUITO
Formamos tu PROPÓSITO DE VIDA

- `404 Not Found` en intentos de consulta, actualización o eliminación de registros no existentes.
- `409 Conflict` cuando se intenta registrar una mascota para un `id_cliente` que no existe en la base de datos (protección de integridad referencial).

5.4. Registro y Matriz de Defectos (Bug Tracking)

Se identificaron 4 hallazgos de severidad media orientados a la ausencia de validaciones de reglas de negocio en los endpoints de inserción y modificación.

Tabla: Bug Tracking:

ID	Módulo	Severidad	Comportamiento Obtenido (Fallo)	Comportamiento Esperado	Estado

DEF-01	Mascotas	Media	La API responde `201 Created` al enviar un payload con `nombre: ""` (vacío).	`400 Bad Request`
				
				`"El nombre es obligatorio".`
					Fixed

DEF-02	Mascotas	Media	La API permite guardar fechas de nacimiento futuras (ej. `2055-01-15`) con `201 Created`.	`400 Bad Request`
				
				`"Fecha inválida".`
					Fixed

DEF-03	Mascotas	Media	El endpoint PUT permite actualizar registros dejando el nombre vacío (`""`) con `200 OK`.	`400 Bad Request` Impedir actualización.	Fixed
DEF-04	Mascotas	Media	El endpoint PUT permite modificar la	`400 Bad Request`	Fixed

(+593) 96 356 1961 admisiones@itq.edu.ec Antonio de Ulloa N28-30 y Diego de Atienza (Esq). WWW.ITQ.EDU.EC

Extracto: Manual Tecnico Vet.pdf original, página 13.